Changes to the Inference Engine

Mestrado em Engenharia de Inteligência Artificial

2024/2025

Team 1 - Neural Networth

José Coelho - 1240148
Luciano Dias - 1121170
Joana Vaz - 1190683
Luís Paula - 1240149
Miguel Rodrigues - 1181734

isep Instituto Superior de Engenharia do Porto

Contents

Operators/Predicates name Redefinition **3**
Operators 3
Predicates 3
Knowledge Base Loading **4**
Fact Processing and Engine Control **4**
Operator 'or' **5**
Addition of new Predicates for enhanced evaluate conditions **6**
New Predicates 6
Operations and Fact Creation **7**

Operators/Predicates name Redefinition

In this update, all operators and predicates have been redefined from Portuguese to English to enhance clarity and standardization across the codebase. This change aims to align the naming conventions with common programming practices and improve the overall readability of the code. The following transformations have been made:

Operators

- '*entao'* has been replaced with '*then'*.
- '*se'* has been replaced with '*if'*.
- '*regra'* has been replaced with '*rule'*.
- '*nao'* has been replaced with '*not'*.
- '*e'* has been replaced with '*and'*.

Predicates

- '*carrega_bc'* has been replaced with '*load_bc'*.
- '*arranca_motor'* has been replaced with '*start_engine'*.
- '*fact_triggers_rules'* has been replaced with '*fact_triggers_rules'*.
- '*facto_dispara_regras1'* has been replaced with '*fact_triggers_rules1'*.
- '*dispara_regras'* has been replaced with '*trigger_rules'*.
- '*facto_esta_numa_condicao'* has been replaced with '*fact_in_condition'*.
- '*verifica_condicoes'* has been replaced with '*verify_conditions'*.
- '*concluir'* has been replaced with '*conclude'*.
- '*cria_facto'* has been replaced with '*create_fact'*.
- '*avalia'* has been replaced with '*evaluate'*.
- '*compara'* has been replaced with '*compare_values'*.
- '*mostra_factos'* has been replaced with '*show_facts'*.
- '*como'* has been replaced with '*how'*.
- '*escreve_factos'* has been replaced with '*write_facts'*.
- '*explica'* has been replaced with '*explain'*.
- '*encontra_regras_whynot'* has been replaced with '*find_whynot_rules'*.
- '*encontra_premissas_falsas'* has been replaced with '*find_false_premises'*.
- '*explica_porque_nao'* has been replaced with '*explain_why_not'*.
- '*formata'* has been replaced with '*format'*.
- '*facto'* has been replaced with '*fact'*.
- '*ultimo_facto'* has been replaced 'with *last_fact'*.
- '*justifica'* has been replaced with '*justifies'*.

These changes not only facilitate a better understanding of the code but also help in maintaining consistency throughout the inference engine.

Knowledge Base Loading

The process of loading the knowledge base has been streamlined. The *'load_kb'* predicate now directly consults three specific files, enabling easier integration of rules and facts. The new structure is as follows:

```
load_kb :-
consult('RealEstateAnalyzer/Expert_System_PPROGIA/Metaknowledge/in
dex.pl'),
consult('RealEstateAnalyzer/Expert_System_PPROGIA/Rules/index.pl')
,
consult('RealEstateAnalyzer/Expert_System_PPROGIA/Facts/index.pl')
.
```

This modification enhances the organization of the knowledge base and simplifies future updates or expansions to the system.

Fact Processing and Engine Control

The new version introduces more efficient fact processing through a loop-based control mechanism in the '*start_engine'*. The '*start_engine_loop'* predicate is now responsible for processing facts incrementally, ensuring that all facts are evaluated before concluding:

Added the '*number_of_facts'* predicate to count the total number of facts in the system. The '*start_engine_loop'* now recursively processes each fact until all have been evaluated. A conditional check has been introduced to signal when all facts have been processed.

```
+ % counts the number of facts
+ number_of_facts(Total) :-
+   findall(N, fact(N, _), LFacts),
+   length(LFacts, Total).
 % start the inference engine
- start_engine:-    fact(N,Fact),
-   fact_triggers_rules1(Fact, LRules),
-   trigger_rules(N, Fact, LRules),
-   last_fact(N).

+ start_engine :-
+   number_of_facts(Total),
+   asserta(last_fact(Total)),
+   start_engine_loop(0, Total).
+ % when all facts are processed
+ start_engine_loop(N, N) :-
+   writeln('All facts have been processed.').
```

```
+ % process each fact and repeat the process after create the new
facts
+ start_engine_loop(N, Total) :-
+   N < Total,
+   N1 is N + 1,
+   fact(N1, Fact),
+   fact_triggers_rules1(Fact, LRules),
+   trigger_rules(N1, Fact, LRules),
+   number_of_facts(Total1),
+   start_engine_loop(N1, Total1).

+restart_engine :-
+    retractall(fact(_, _)),        % Remove all facts
+    retractall(last_fact(_)),      % Reset the fact counter
+    retractall(justifies(_, _, _)). % Remove justifications
```

This restructuring improves the engine's responsiveness and ensures that all facts are accurately processed before the conclusion phase.

Operator 'or'

In addition to the changes in operator names, a new operator *'or'* has been introduced. This operator allows for more flexible logical expressions and expands the capabilities of the inference engine by enabling disjunctive conditions in rule definitions. The inclusion of the or operator facilitates complex decision-making processes and enhances the expressiveness of the rules defined within the system.

```
 fact_in_condition(F,[F  and _]).
+ fact_in_condition(F,[F  or _]).
 fact_in_condition(F,[evaluate(F1)  and _]):-
F=..[H,H1|_],F1=..[H,H1|_].
+ fact_in_condition(F,[evaluate(F1)  or _]):-
F=..[H,H1|_],F1=..[H,H1|_].
 fact_in_condition(F,[_ and Fs]):- fact_in_condition(F,[Fs]).
+ fact_in_condition(F,[_ or Fs]):- fact_in_condition(F,[Fs]).

 verify_conditions([not evaluate(X) and Y],[not X|LF]):- !,
   \+ evaluate(_,X),
   verify_conditions([Y],LF).

 verify_conditions([evaluate(X) and Y],[N|LF]):- !,
   evaluate(N,X),
   verify_conditions([Y],LF).

 verify_conditions([not X and Y],[not X|LF]):- !,
```

```
   \+ fact(_,X),
   verify_conditions([Y],LF).

 verify_conditions([X and Y],[N|LF]):- !,
   fact(N,X),
   verify_conditions([Y],LF).

+ verify_conditions([not evaluate(X) or Y],[not X|LF]):- !,
+   \+ evaluate(_,X),
+   verify_conditions([Y],LF).

+ verify_conditions([evaluate(X) or Y],[N|LF]):- !,
+   (evaluate(N,X) ; verify_conditions([Y],LF)).

+ verify_conditions([not X or Y],[not X|LF]):- !,
+   (\+ fact(_,X) ; verify_conditions([Y],LF)).

+ verify_conditions([X or Y],[N|LF]):- !,
+   (fact(N,X) ; verify_conditions([Y],LF)).
```

we change the weight of the precedence between if and or , in order to adjust the way our engine , the conditions with or should run before the and operator

```
:-op(220,xfx,then).
:-op(35,xfy,if).
:-op(240,fx,rule).
:-op(500,fy,not).
+:-op(700,xfy,and).
:-op(600,xfy,or).
```

Addition of new Predicates for enhanced evaluate conditions

To further enhance the flexibility and expressiveness of the inference engine, two new predicates have been added. These predicates allow the creation of rules that utilize the evaluate predicate either alone or in combination with multiple evaluate conditions connected by the and operator. This enhancement facilitates more complex rule definitions and improves the engine's decision-making capabilities.

New Predicates

- *verify_conditions([evaluate(X)], _)*: Allows rules to include a single evaluate condition.

- *verify_conditions([evaluate(X) and evaluate(Y)], _)*: Enables rules to incorporate multiple evaluate conditions connected by the and operator.

```
 % verifies conditions
+ verify_conditions([evaluate(X)],_):-
+   evaluate(_,X).
+
+ verify_conditions([evaluate(X) and evaluate(Y)],_):-
+   evaluate(_,X), evaluate(_,Y).
  verify_conditions([not evaluate(X) or Y],[not X|LF]):- !,
   \+ evaluate(_,X),
   verify_conditions([Y],LF).
  verify_conditions([evaluate(X) or Y],[N|LF]):- !,
   (evaluate(N,X) ; verify_conditions([Y],LF)).
  verify_conditions([not X or Y],[not X|LF]):- !,
   (\+ fact(_,X) ; verify_conditions([Y],LF)).
  verify_conditions([X or Y],[N|LF]):- !,
   (fact(N,X) ; verify_conditions([Y],LF)).
  verify_conditions([not evaluate(X) and Y],[not X|LF]):- !,
   \+ evaluate(_,X),
   verify_conditions([Y],LF).
  verify_conditions([evaluate(X) and Y],[N|LF]):- !,
   evaluate(N,X),
   verify_conditions([Y],LF).
  verify_conditions([not X and Y],[not X|LF]):- !,
   \+ fact(_,X),
   verify_conditions([Y],LF).
  verify_conditions([X and Y],[N|LF]):- !,
   fact(N,X),
   verify_conditions([Y],LF).
  verify_conditions([not X],[not X]):- !, \+ fact(_,X).
 verify_conditions([X],[N]):- fact(N,X).
```

Operations and Fact Creation

This chapter introduces the ability to create facts based on operations passed as parameters. New operations such as counting, multiplication, and summing values across facts have been added. The system now supports receiving facts with either two or three elements, and these can be used in conjunction with operations to generate new facts.

```
  create_fact(F,_,_):-
   fact(_,F),!.

+ create_fact(F,ID,LFacts):-
+   F=..[V1,V2,Z],
+   Z=..[operation,O,X,L],
```

```
+   operate(O,X,L,Result),
+   FinalFact =..[V1,V2,Result],
+   create_fact(FinalFact,ID,LFacts).

+ create_fact(F,ID,LFacts):-
+   F=..[V1,V2,V3,Z],
+   Z=..[operation,O,X,L],
+   operate(O,X,L,Result),
+   FinalFact =..[V1,V2,V3,Result],
+   create_fact(FinalFact,ID,LFacts).

  create_fact(F,ID,LFacts):-
   writeln('fact'),
   retract(last_fact(N1)),
   N is N1+1,
   asserta(last_fact(N)),
+  flatten_list(LFacts, FlatLFacts),
+  assertz(justifies(N,ID,FlatLFacts)),
   assertz(fact(N,F)),!.

+ operate(multi,X,L,Result):-
+  multiplication(X,L,Result).

+ operate(count,X,L,Result):-
+   count(X,L,Result).

+ operate(sum_all,X,L,Result):-
+   sum_all(X,L,Result).

+ multiplication(X, Y, Result) :-
+   Result is X * Y.

+ sum_all(X, _, Result) :-
+   X=..[_,_,_,V],
+   findall(V, fact(_,X), Values),
+   sum_list(Values, Result).

+ count(X, _, Result) :-
+   findall(_, fact(_, X), LFacts),
+   length(LFacts, Result).

+ flatten_list([], []).

+ flatten_list([H|T], FlatList) :-
+    flatten_list(H, FlatH),
+    flatten_list(T, FlatT),
+    append(FlatH, FlatT, FlatList).
```

```
+ flatten_list(L, [L]) :-
+     \+ is_list(L).
```

we noticed that the conditions there evaluated did not have the previous rule. Due to that the list of that stores the justification facts had anonymous variable for those facts , what was causing the infinity loop on the explain of the facts. so we created the *flatten_list* to remove the cases where a list had anonymous variables.